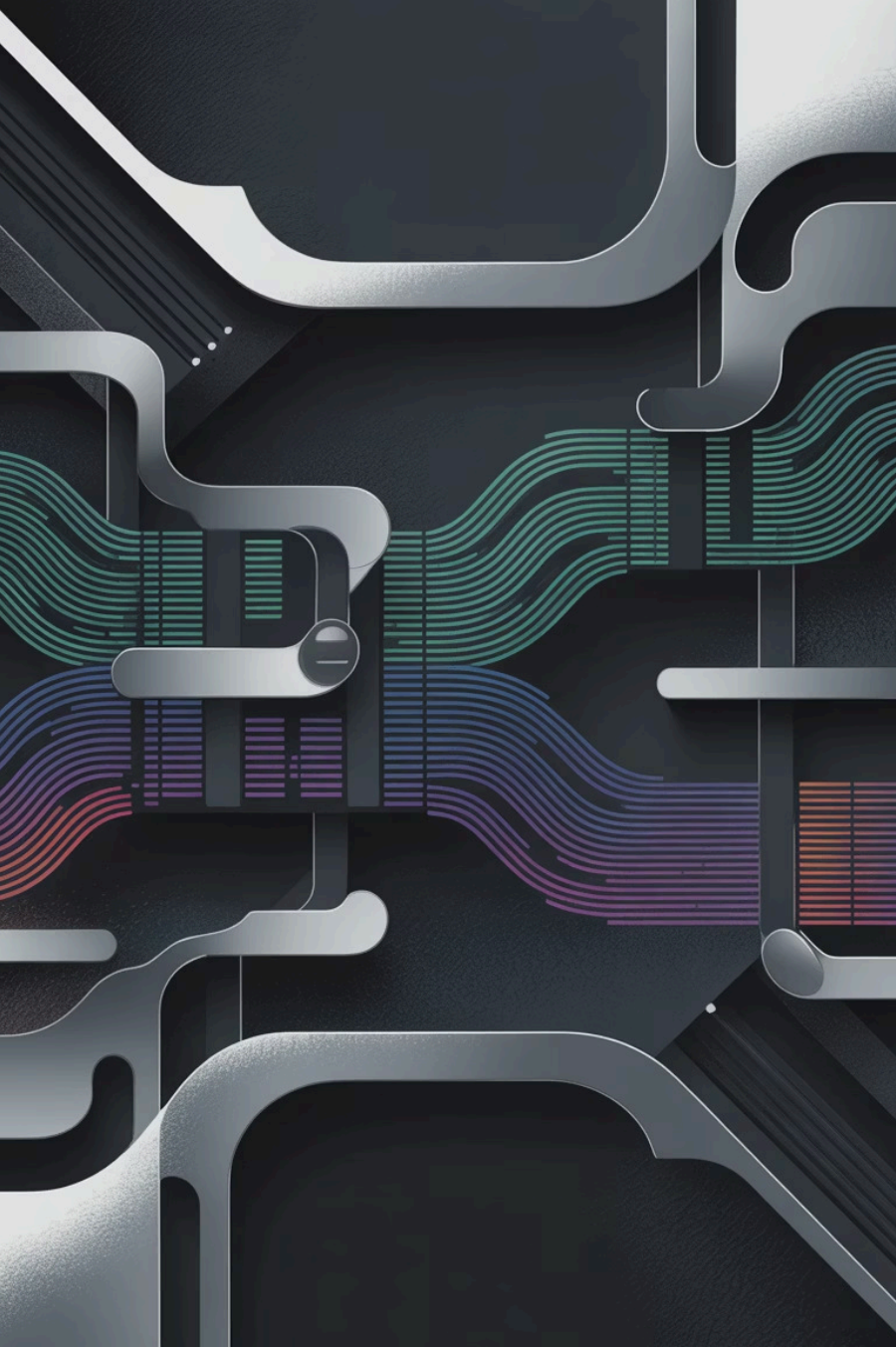




Module 11- Data Engineering for LLM Pipelines

Building the Foundation for AI Engineering Excellence

Learning Objectives



Schema Fundamentals

Master the core instruction tuning structures and understand when to apply {instruction, input, output} versus {prompt, completion} patterns.



Templating Strategies

Learn to construct effective prompt templates using delimiter patterns, role markers, and structural conventions that improve model comprehension.



Validation Pipelines

Implement schema validation using Pydantic models and data profiling techniques to ensure dataset quality before training.

Why Instruction Tuning Schemas Matter

The Challenge

Pre-trained language models possess broad knowledge but struggle with task-specific behaviors. Raw training data lacks the structured format needed to teach models how to follow instructions, respond appropriately, and maintain consistent output patterns.

Without proper schema design, fine-tuning efforts fail to generalize across diverse prompts, produce unreliable outputs, and require excessive training iterations.

The Solution

Well-designed instruction schemas provide the structural foundation that enables models to learn task patterns efficiently. Consistent formatting, clear role delineation, and standardized templates create training signals that improve instruction-following capabilities.

Schema-driven approaches reduce training time, improve output quality, and enable systematic validation of dataset quality before expensive training runs.

Core Schema Architectures

Two primary schema patterns dominate instruction tuning workflows, each optimized for different use cases and model architectures. Understanding their structural differences guides appropriate schema selection.

`{instruction, input, output}`

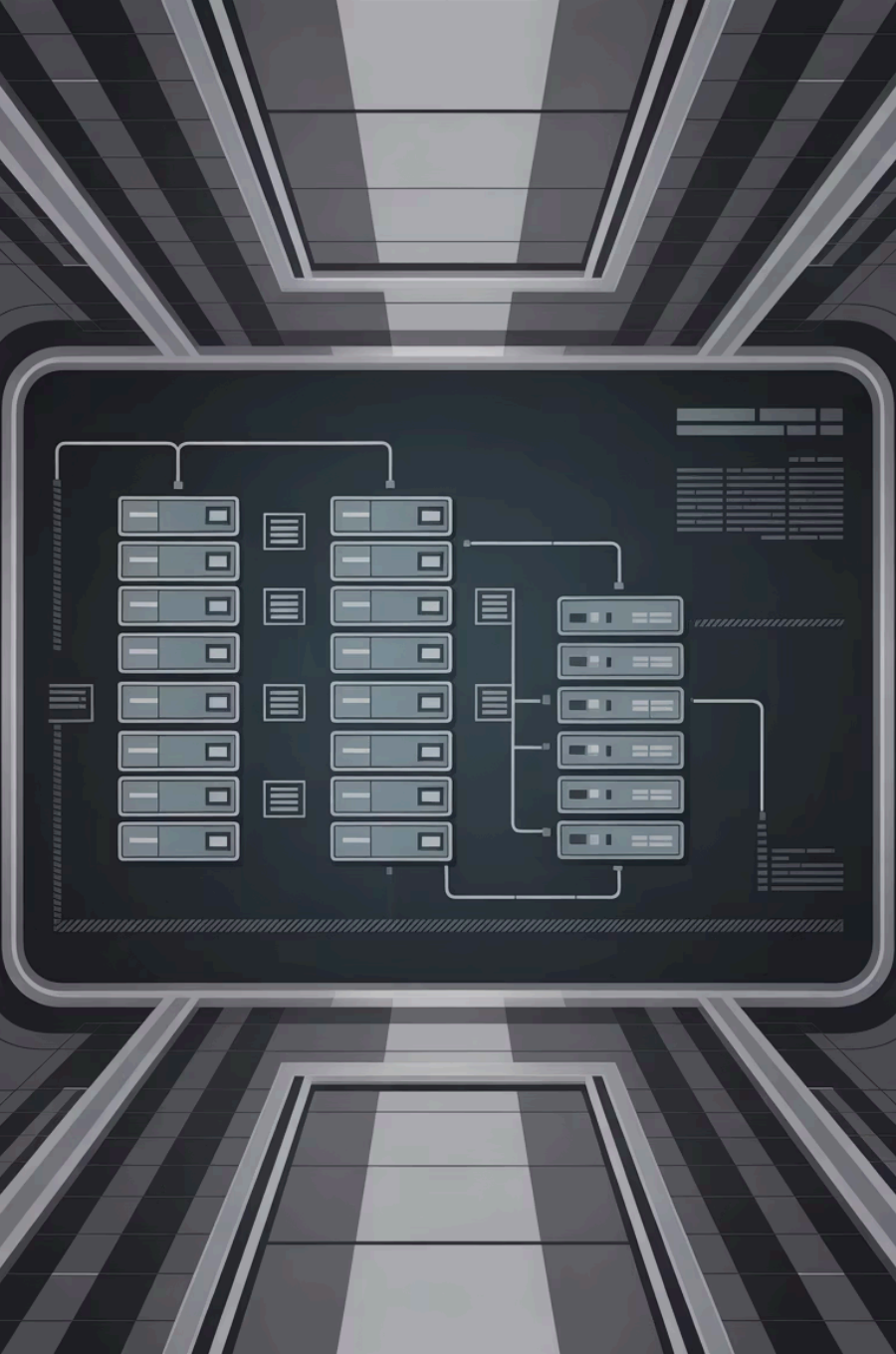
Three-field schema separating task description, context, and expected response. Optimal for complex tasks requiring explicit instruction and variable input context.

`{prompt, completion}`

Two-field schema combining instruction and input into a single prompt field. Streamlined for simpler tasks and models expecting concatenated prompt-response pairs.

Legacy two-field format. Still seen in older tutorials and some OSS trainers; superseded by role-based messages for chat fine-tuning.

The `{prompt, completion}` framework is shown here but, for OpenAI tuning, it is considered "legacy". Provided for completeness.



The {instruction, input, output} Pattern

```
{  
  "instruction": "Summarize the following research abstract",  
  "input": "Recent advances in transformer architectures have...",  
  "output": "This paper presents improvements to transformer..."  
}
```

This three-component schema explicitly separates the task directive from contextual input and expected output. The instruction field defines the task, input provides context or data to process, and output demonstrates the desired response format.

This pattern excels in scenarios requiring clear task-context separation: document summarization, question answering with context passages, translation tasks, and code generation from specifications.

When to Use {instruction, input, output}



Variable-length context handling

Tasks where input context varies significantly in length benefit from explicit separation. Research paper summarization, document analysis, and long-form content transformation require this clarity.



Multi-step reasoning tasks

Complex tasks involving analysis, synthesis, or transformation of input data work better when instructions are isolated from the material being processed.



Dataset consistency requirements

When maintaining uniform structure across diverse task types within a single training corpus, three-field schemas provide clearer boundaries and easier validation.

The {prompt, completion} Pattern

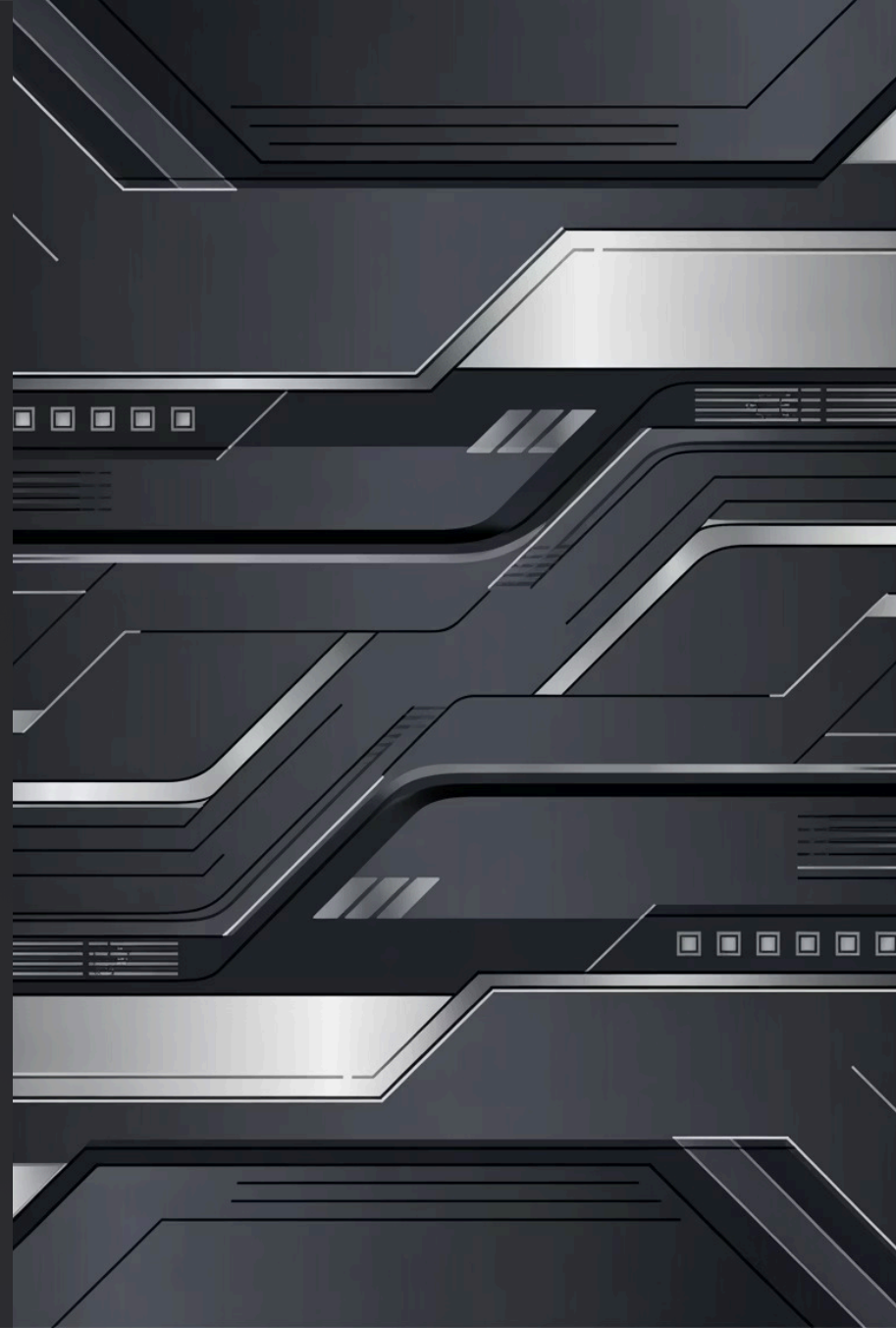
```
{  
  "prompt": "Generate a product description for a noise-canceling headphone",  
  "completion": "Experience immersive audio with our premium noise-canceling..."  
}
```

This streamlined two-field schema combines instruction and context into a single prompt field, paired with the corresponding completion. The approach mirrors the natural input-output flow of language model inference.

Originally popularized by OpenAI's fine-tuning APIs, this pattern works well for straightforward generation tasks, conversational AI training, and scenarios where instruction and context naturally blend together. **That original API is now legacy — today's OpenAI fine-tuning is chat/messages-based (slide 25).**

Prompt Templating Fundamentals

Effective templating transforms raw schema fields into formatted prompts that models can reliably parse and respond to. Consistent delimiter patterns, role markers, and structural conventions significantly impact model performance during fine-tuning and inference.



Delimiter-Based Templating Patterns

```
### Instruction:
```

```
{instruction}
```

```
### Input:
```

```
{input}
```

```
### Response:
```

```
{output}
```

Delimiter-based templates use clear visual markers to separate schema components. The `### Instruction:` pattern has become a de facto standard, providing models with explicit boundaries between task description, context, and expected output.

These delimiters serve as structural anchors during training, helping models learn to distinguish between instructions to follow versus content to process. Consistency across your entire training dataset is critical for reliable learning.

Alternative Templating Approaches



Chat-Style Templates

Using role markers like "User:" and "Assistant:" to create conversational training data. Particularly effective for chatbot fine-tuning and multi-turn dialogue systems.

```
User: {instruction}\nAssistant:  
{output}
```



XML-Style Delimiters

Structured markup using tags like `<instruction>` and `</instruction>` for precise field boundaries. Reduces ambiguity in complex nested content scenarios.

```
<instruction>{text}</instruction>
```



Special Token Markers

Reserved tokens like `[INST]`, `[/INST]`, or model-specific markers (e.g., Llama's `<s>`, `</s>`). Integrates with tokenizer vocabulary for efficient processing.

```
[INST] {instruction} [/INST] {output}
```

Template Design Principles

01

Consistency Above All

Use identical delimiters, spacing, and formatting patterns across every training example. Inconsistent templates create noise that degrades model learning.

03

Avoid Natural Language Collisions

Choose delimiters unlikely to appear in actual content. "####" is safer than ":" which commonly appears in text, reducing parsing ambiguity.

02

Clear Visual Separation

Employ newlines, whitespace, or distinctive markers between sections. Models learn structural patterns more effectively when boundaries are visually obvious.

04

Align with Inference Patterns

Design templates that match your production inference format exactly. Training-inference mismatch creates distribution shift that hurts performance.

Practical Example: Text Classification Task

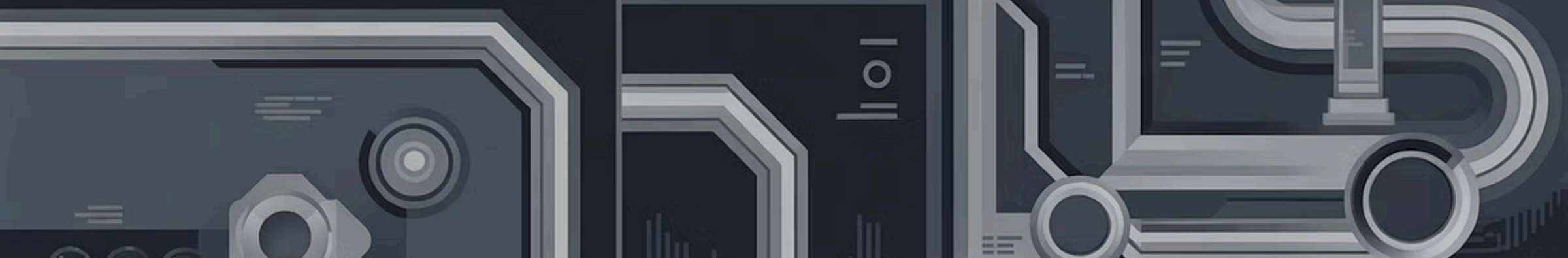
Raw Schema Data

```
{  
  "instruction": "Classify sentiment",  
  "input": "This product exceeded expectations",  
  "output": "positive"  
}
```

Templated Training Format

```
### Instruction:  
Classify the sentiment of the  
following text as positive,  
negative, or neutral.  
  
### Input:  
This product exceeded expectations  
  
### Response:  
positive
```

The template expands the terse instruction into clear task framing, adds explicit formatting expectations, and creates visual structure that improves model comprehension during training.



Dataset Transformation Pipeline Architecture



Raw Data

Source JSON, CSV, or API responses with unstructured fields



Schema Validation

Pydantic models enforce field presence, types, constraints



Template Application

Format fields using consistent delimiter patterns



Training Format

JSONL with text field ready for tokenization

Pydantic Schema Validation Models

```
from pydantic import BaseModel, Field, field_validator

class InstructionSchema(BaseModel):
    instruction: str = Field(..., min_length=10, max_length=500)
    input: str = Field(default="")
    output: str = Field(..., min_length=1, max_length=2000)

    @field_validator("instruction")
    @classmethod
    def validate_instruction(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("Instruction cannot be empty")
        return v.strip()
```

Pydantic models provide type checking, field validation, and automatic error reporting. The Field constraints enforce length requirements while custom validators handle domain-specific rules like whitespace normalization.

Why Pydantic for Schema Validation?

Type Safety

Automatic type coercion and validation prevents silent data corruption. Catches string-integer mismatches, missing fields, and null values before they enter training pipelines.

Self-Documenting

Schema definitions serve as living documentation. Field descriptions, constraints, and examples are embedded directly in validation code.

Error Aggregation

Validates entire records and reports all errors simultaneously rather than failing on first issue. Accelerates dataset debugging cycles.

JSON Compatibility

Native JSON serialization with `.model_dump()` (→ **dict) and `.model_dump_json()`.** Seamlessly integrates with API endpoints and file-based workflows.

Advanced Validation Patterns

```
from pydantic import BaseModel, model_validator

class InstructionDataset(BaseModel):
    instruction: str
    input: str = ""      # default -> consistent with slide 15, no longer "required"
    output: str

    @model_validator(mode="after")
    def check_cross_fields(self):
        # Reject substantive instructions paired with a trivially short output
        if len(self.instruction) > 100 and len(self.output) < 20:
            raise ValueError("Output too short for complex instruction")
        return self
```

Cross-field validators enable sophisticated quality checks that consider relationships between schema components, catching subtle consistency issues.

Implementing the Transformation Pipeline

```
def transform_to_training_format(raw_data: List[dict]) -> List[dict]:
    """Transform validated schemas into templated training examples."""
    training_examples = []

    for record in raw_data:
        # Validate against schema
        validated = InstructionSchema(**record)

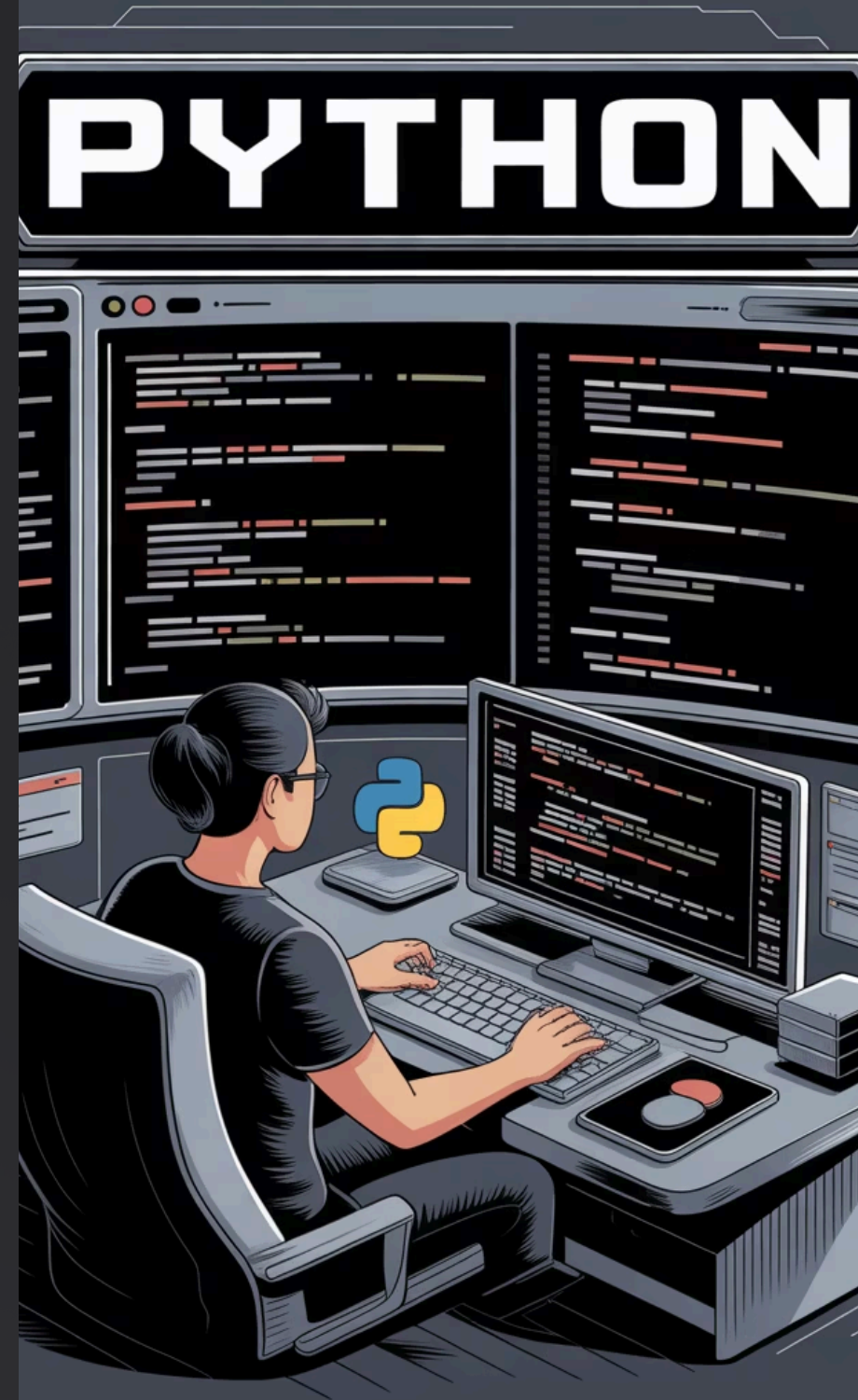
        # Apply template
        prompt = f"""### Instruction:
{validated.instruction}

### Input:
{validated.input}

### Response:
{validated.output}"""

        training_examples.append({
            "text": prompt,
            "metadata": {"source": record.get("source", "unknown")}
        })

    return training_examples
```



Data Quality Profiling Techniques



Length Distribution Analysis

Profile token counts for instructions, inputs, and outputs. Identify outliers that may cause training instability or exceed context windows.



Vocabulary Coverage

Analyze unique tokens and vocabulary overlap between training examples. Detect overly repetitive patterns or insufficient diversity.



Duplicate Detection

Identify exact or near-duplicate examples that inflate dataset size without adding learning signal. Use fuzzy matching for semantic similarity.



Label Distribution

For classification tasks, ensure balanced representation across output classes. Severe imbalance degrades minority class performance.

Profiling with Pandas and Custom Metrics

```
import pandas as pd

def profile_dataset(examples: List[dict]) -> pd.DataFrame:
    """Generate quality metrics for instruction dataset."""

    df = pd.DataFrame(examples)

    # Token length approximations (whitespace-based)
    df['instruction_tokens'] = df['instruction'].str.split().str.len()
    df['input_tokens'] = df['input'].str.split().str.len()
    df['output_tokens'] = df['output'].str.split().str.len()
    df['total_tokens'] = df[['instruction_tokens', 'input_tokens',
                            'output_tokens']].sum(axis=1)

    # Statistical summary
    summary = df[['instruction_tokens', 'input_tokens',
                  'output_tokens', 'total_tokens']].describe()

    return summary
```

Basic profiling reveals distribution characteristics that guide preprocessing decisions like truncation thresholds and batching strategies.

Common Schema Validation Failures

Empty or Null Fields

Missing required fields like empty instructions or null outputs break training. Pydantic's `Field(...)` syntax enforces presence validation.

Excessive Length

Outputs exceeding model context windows cause truncation during training. Set `max_length` constraints aligned with your target model's capabilities.

Encoding Issues

Unicode characters, special symbols, or binary data embedded in text fields. Normalize to UTF-8 and sanitize non-printable characters.

Inconsistent Formatting

Mixed use of markdown, HTML tags, or code blocks across examples. Standardize formatting conventions during preprocessing.

Handling Optional Input Fields

Input-Required Tasks

```
{  
  "instruction": "Summarize the text",  
  "input": "Machine learning is...",  
  "output": "ML enables systems..."  
}
```

Template:

Instruction:

Summarize the following text.

Input:

Machine learning is...

Response:

ML enables systems...

Input-Optional Tasks

```
{  
  "instruction": "Write a haiku about AI",  
  "input": "",  
  "output": "Silicon thinking..."  
}
```

Template:

Instruction:

Write a haiku about AI.

Response:

Silicon thinking...

Conditional template logic omits the Input section when the field is empty, maintaining clean formatting without unnecessary delimiters.

Flexible Template Function Implementation

```
def apply_template(instruction: str, output: str, input_text: str = "") -> str:
    """Apply instruction template with optional input field."""

    template_parts = [f"### Instruction:\n{instruction}"]

    # Only include input section if content exists
    if input_text and input_text.strip():
        template_parts.append(f"### Input:\n{input_text}")

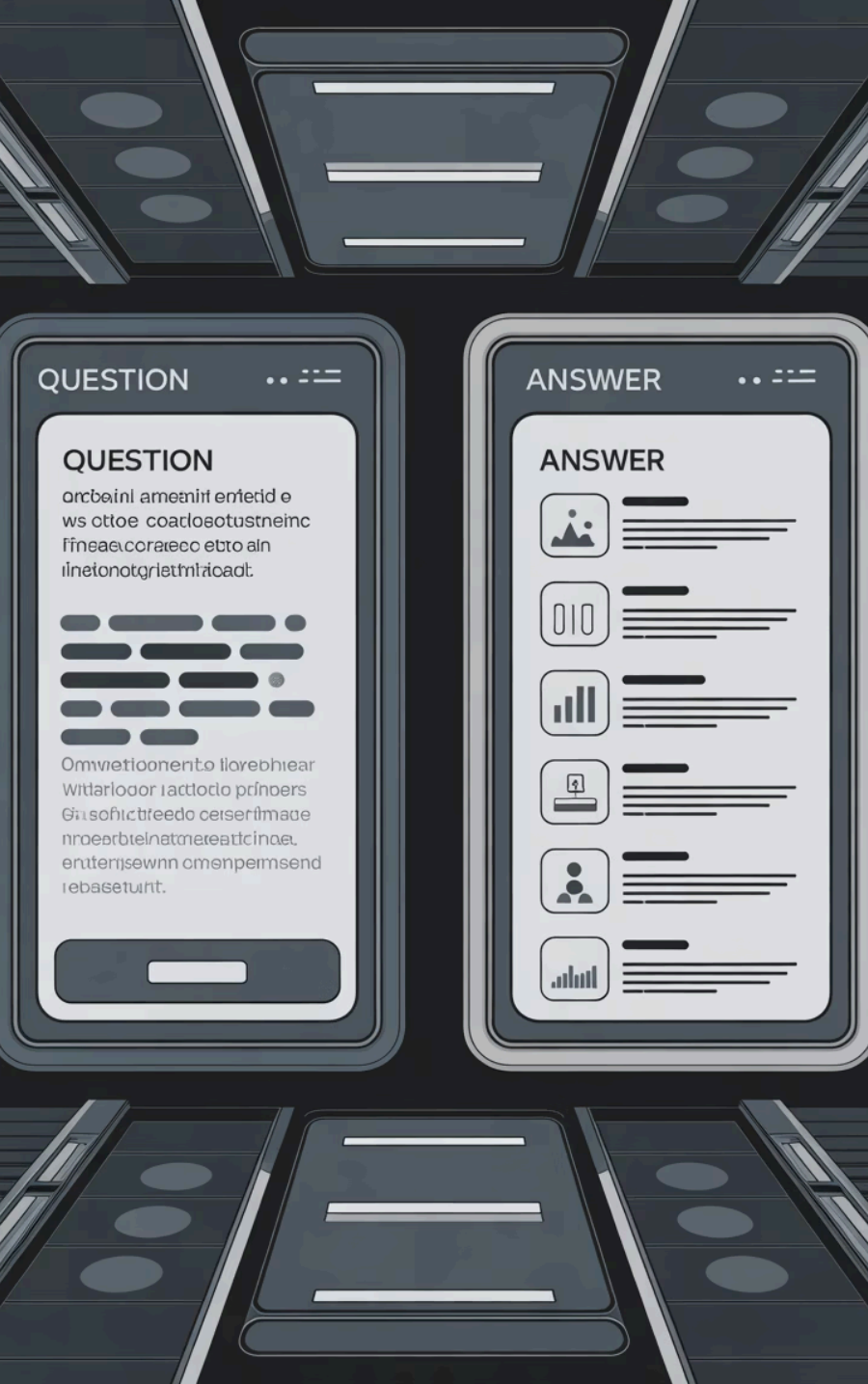
    template_parts.append(f"### Response:\n{output}")

    return "\n\n".join(template_parts)

# Example usage
prompt1 = apply_template(
    instruction="Classify sentiment of the text",
    input_text="This product is amazing",
    output="positive"
)

prompt2 = apply_template(
    instruction="Generate a creative product name for headphones",
    output="SoundScape Pro"
)
```

Real-World Example: Question Answering Dataset



```
# Raw data from SQuAD-style dataset
```

```
raw_example = {  
    "context": "The transformer architecture was introduced in 2017...",  
    "question": "When was the transformer architecture introduced?",  
    "answer": "2017"  
}
```

```
# Transform to instruction schema
```

```
instruction_example = {  
    "instruction": "Answer the question based on the provided context.",  
    "input_text": f"Context: {raw_example['context']}\n\n",  
    "Question": f"Question: {raw_example['question']}",  
    "output": raw_example["answer"]  
}
```

```
# Apply template (keys now match apply_template's parameters)
```

```
training_text = apply_template(**instruction_example)
```

This transformation maps domain-specific schemas to standardized instruction formats, enabling consistent training across diverse data sources.

Export Formats for Fine-Tuning Frameworks

Hugging Face SFT — templated single-string field (current)

```
{"text": "### Instruction:\nClassify sentiment...\n\n### Response:\npositive"}
```

prompt/completion — **LEGACY** (older **OpenAI** + some **OSS** trainers)

```
{"prompt": "### Instruction:\nSummarize...", "completion": "The article discusses..."}
```

OpenAI fine-tuning — role-based messages (current)

```
{"messages": [  
  {"role": "system", "content": "You are a helpful assistant"}, # system optional  
  {"role": "user", "content": "Summarize this text..."},  
  {"role": "assistant", "content": "The summary is..."}  
]}
```

Different frameworks expect specific schema variations. Understanding these formats enables portability across training platforms and model architectures.

Best Practices for Production Datasets

Version Control Your Schemas

Track Pydantic model definitions in Git alongside code. Schema evolution requires careful versioning to maintain reproducibility of training runs.

1

2

Separate Validation from Transformation

Run validation as a distinct pipeline stage before templating. This separation enables reprocessing with different templates without re-validating source data.

Log Validation Failures

Capture rejected records with detailed error messages for debugging. Aggregate failure patterns to identify systematic data quality issues.

3

4

Maintain Raw and Processed Artifacts

Store both validated schemas and templated outputs. This enables template iteration without revalidating, and revalidation without reacquiring source data.



Scaling Validation Pipelines

As datasets grow to millions of examples, single-threaded validation becomes a bottleneck. Parallel processing strategies significantly accelerate preprocessing workflows.

Python's multiprocessing module enables CPU-bound validation across multiple cores. For larger datasets, frameworks like Apache Spark or Dask provide distributed processing capabilities.

Batch validation in chunks of 1000-10000 records balances memory efficiency with parallelization overhead. Stream processing from data lakes using tools like Delta Lake enables incremental validation of continuously updating datasets.

Performance Considerations

- Profile validation bottlenecks using cProfile
- Cache compiled regex patterns for repeated validation
- Use bulk database operations for source data extraction
- Implement checkpointing for long-running pipelines

Key Takeaways



Schema Choice Matters

Select {instruction, input, output} for complex tasks requiring context separation. Use {prompt, completion} for simpler generation tasks and API compatibility.



Template Consistency is Critical

Apply identical delimiter patterns, spacing, and formatting across all training examples. Training-inference format alignment is non-negotiable.



Validate Early and Often

Use Pydantic schemas to catch data quality issues before training. Profile datasets to understand distribution characteristics and identify outliers.



Thanks